scientific reports

Check for updates

OPEN

Real-time multiple people gait recognition in the edge

Paula Ruiz-Barroso, José María González-Linares✉, Francisco M. Castro & Nicolás Guil

Deploying deep learning models on edge devices offers advantages in terms of data security and communication latency. However, optimizing these models to achieve fast computing speeds without sacrificing accuracy can be challenging, especially in video surveillance applications where real-time processing is crucial. In this study, we investigate the deployment of gait recognition models as a multi-objective selection problem in which we seek to simultaneously minimize several objectives, such as latency and energy consumption, while maintaining accuracy. The decision space of a problem comprises all models that can be built by varying parameters, such as the size of the model, the operating frequency of the device, and the precision of the operations. From this problem definition, a subset of Pareto optimal models can be selected to be deployed on the target device. We conducted experiments with two different gait recognition models on NVIDIA Jetson Orin Nano and Jetson AGX Orin to explore their decision spaces. In addition, we investigated different strategies to increase the throughput of the deployed models by taking advantage of batching and concurrent execution. Together, these techniques allowed us to design real-time solutions for gait recognition in scenarios with multiple subjects. These solutions can process between 42 and 188 simultaneous subjects at 25 inferences per second with an energy consumption ranging from 6.31 to 9.71 mJ per inference, depending on the device and the deployed model.

Edge computing offers several general advantages, including reduced latency, improved bandwidth efficiency, enhanced data privacy, and real-time processing capabilities, by bringing computation closer to the data source. These benefits are critical in many applications, such as drone navigation[1] and real-time video analytics[2].

Many efforts have been made to deploy deep learning-based real-time video analytics applications on edge devices, such as intelligent surveillance or autonomous driving[3]. Gait, which is akin to biometric features like fingerprints or iris patterns[4], enables identification through an individual's unique walking style. Extensively studied from medical[5,6], human–computer interaction[7,8], and sports performance perspectives[9], gait serves as a pivotal biometric feature for person identification, as evidenced by numerous prior studies[10]. Unlike other biometric methods, gait identification does not require direct cooperation with the system and can be performed remotely. As a result, significant research has focused on gait recognition, using various data inputs like silhouettes[11], gait energy images[12], optical flow[13], and inertial sensors[14]. The rise of deep learning has advanced gait recognition techniques to unprecedented levels of accuracy. However, current methods still fail to meet the demands of practical surveillance systems, which require real-time feature extraction and processing[15].

Moving the computation of deep learning applications to the edge can help to develop more scalable and robust applications, as no data communication with cloud servers is required[16]. Thus, reliability and data privacy are improved, and communication latency is reduced. In addition, deploying deep learning applications on edge devices conveys a trade-off between the need for high performance, both in terms of accuracy and latency, and low power consumption. These deep learning applications can be adapted using techniques such as quantization[17], pruning[18] or model compression[19]. In addition, edge hardware platforms offer architectural innovations, such as different types of processing units or various operating frequencies. While leveraging all these features can be beneficial, it is often a complicated task.

Further enhancements to the underlying system architecture can also be implemented. Specifically, batching and stream processing techniques can be used on GPUs[20,21]. Batching involves grouping multiple independent inferences that can be executed in parallel, whereas streams facilitate the concurrent execution of inferences on GPUs, provided that hardware resources are available. Although both techniques can improve application throughput, they can also adversely affect inference latency. In addition, current edge devices have various operating frequencies. Lowering the frequency can reduce the energy consumption; however, it can negatively affect the latency.

Figure 1 shows a motivating example where two video surveillance cameras monitor the flow of people through the entrance hall of a building. Edge devices may detect and track people, and an edge server receives all

Department of Computer Architecture, Institute for Mechatronics Engineering & Cyber-Physical Systems (IMECH.UMA), Universidad de Málaga, Málaga, Spain. ✉email: jgl@uma.es

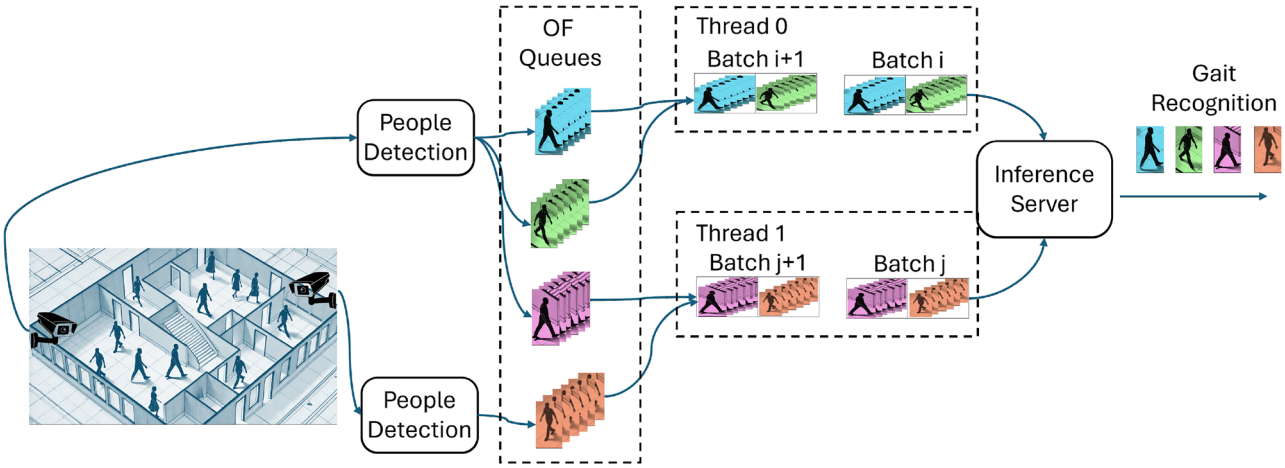


Fig. 1. Concurrent deployment scenario. Two cameras recording people walking through a hall. The optical flow of the detected people is pushed in FIFO queues. Concurrency is achieved by launching several threads in which each thread processes a batch of optical flow maps of several persons.

such detections and recognizes people by gait using different models to adapt to the workload and required QoS. In such a scenario, the flow of people can be quite variable; thus, the system must be able to adapt to different workloads using both sample batching and inference concurrency.

In this work, we leverage two embedded heterogeneous architectures as edge servers, specifically, an NVIDIA Jetson Orin Nano and Jetson AGX Orin, to deploy multiple gait recognition models. These heterogeneous platforms have different computing capabilities, which allowed us to study the scalability of our solution. Furthermore, we evaluated advanced techniques for deploying deep learning models, such as batching and concurrency, by exploiting the integrated GPU available on both platforms. In addition, to select the best models, the proposed approach considers several desirable objectives, such as minimizing latency and energy consumption while aiming for high accuracy. This selection process is performed using a multiobjective optimization framework that simultaneously considers all objectives within a decision space. The decision space comprises a feasible set of decision vectors, including parameters such as the pruning factor, GPU frequency, and data precision. Other studies have tried to dynamically adapt the operational accuracy of a model to optimize some objective[22,23]. In these studies, they selected Pareto optimal models for only two objectives: accuracy and a second objective (energy, memory, etc.), which is a clear limitation compared to our approach, that deals with a higher dimensional Pareto front. Moreover, unlike previous work[2] that carried out simple studies about parallel execution of deep learning inferences, in this work we performed a thorough evaluation of the impact of batching and the concurrent execution of multiple inferences simultaneously using multiple streams.

Thus, the main contributions of this paper can be summarized in:

- A gait recognition system that achieves real-time performance in an NVIDIA Jetson Orin Nano and NVIDIA Jetson Orin AGX by recognizing between 42 and 188 simultaneous subjects at 25 inferences per second with energy consumption ranging from 6.31 to 9.71 millijoules per inference, depending on the device and the deployed model.
- A comprehensive analysis combining energy consumption, latency, and accuracy to evaluate the deployment of two gait recognition models on an embedded platform. We consider different optimization techniques such as pruning and quantization, and study their effects on the inference performance of edge devices.
- A thorough study of selecting optimal gait recognition models using multiobjective optimization considering latency, accuracy, and energy consumption to obtain a subset of Pareto optimal solutions in a decision space. The decision space includes optimization techniques, such as pruning or quantization, that lead to a set of more than one thousand solutions. The proposed framework can be extended to include other optimization objectives and techniques.
- An in-depth study of the benefits of using more than one sample per inference (batch) and concurrency in the deployment of models belonging to the Pareto optimal front to cope with situations where gait recognition must be applied to multiple simultaneous subjects.

To the best of our knowledge, no previous studies on gait recognition at the edge have considered a multiobjective approach for selecting the best deep learning models or studied the impact of concurrent inferences on throughput and energy consumption.

The remainder of this paper is organized as follows. “Related work” summarizes the related work. In “Model description” we describe the input data, the chosen models, and proposed model selection. Next, in “Experiments” we discuss the experimental results, and finally, “Conclusions” concludes the paper.

Related work

Recently, the deployment of deep learning models on embedded devices has gained significant importance. Commonly, these systems are utilized solely for inference owing to their restricted computational capacities[24–26]. In the following, we discuss related work involving general optimization techniques that are applied to the deployment of deep learning models on edge devices. In addition, we review and compare gait recognition solutions implemented on the edge.

Deployment of deep learning models in the edge

Many efforts have been devoted to enhancing inference throughput on embedded devices by implementing pruning or quantization techniques, methods for selecting optimal models, and strategies for concurrent computation.

Various techniques can be employed with deep learning models to mitigate both the computational and energy consumption demands. Knowledge distillation involves compressing a larger 'teacher' model into a smaller 'student' model while maintaining similar accuracy levels[19]. Pruning techniques[18] focus on reducing the number of arithmetic operations by eliminating filters or entire layers from the model. Various pruning methods exist, including weight-by-weight, filter-by-filter, and layer-by-layer approaches. Thus,[27] introduces a network pruning method that detects structural redundancy in a CNN by establishing a graph for each layer and employing two graph-related measures to identify and prune redundant layer filters. AutoPruner[28] is a channel selection layer that can be appended to any convolutional layer. The proposed method identifies less significant filters during training and automatically prunes them.

Additionally, simpler data representation methods, like quantization[17], can be utilized to reduce precision, such as employing 16-bit floating-point or 8-bit integer data instead of the standard 32-bit floating-point representation during inference. The use of lower numerical representations is notably useful in order to reduce both the execution time and energy consumption. However, note that modifying the original model or its parameters can alter the accuracy of the model. There exist different quantization techniques that can be applied after[2,29] or during[30,31] model training to reduce the numerical representation of models. Even though post-training quantization is easier to implement, it usually has the consequence of degrading the accuracy of the model, whereas applying during-training quantization is more computationally intensive but enhances model performance with regard to post-training quantization. In[32], two approaches were proposed to optimize the training process via a quantization procedure. The first method involves incorporating trainable scale thresholds per filter, while the second method is centered around mutual re-scaling of consecutive depthwise separable convolutions and normal convolutions.

As previous techniques can generate several versions of the same model, some studies have attempted to find an optimal subset of models depending on different goals defined by hardware and users using a Pareto front for only two objectives. Minhas et al.[22] use accuracy and an additional objective, such as power, memory, execution time, or combinations of them. Thus, they have a small search space for selected parameters against accuracy. Scheidegger et al.[23] follow the same strategy and created the Pareto optimal front using only the data precision and model architecture.

Finally, several authors have suggested concurrently using the available processing elements to reduce the inference time of deep learning models. These studies used batching or concurrency to compute several inferences at the same time. In[33] the problem of computing the optimal batch size was studied, but they omitted the use of concurrency. In[34] the authors evaluated AI multi-tenancy using concurrent model execution and dynamic model placement. They explored the limits of concurrency and batching to maximize throughput, but did not take energy consumption into account.

Gait recognition in the edge

There are also a few works that have developed gait recognition applications for edge devices. Thus, in[35] a gait identification model that works with inertial data is deployed on a mobile device. The model consists of a CNN and an LSTM working in tandem, but optimizations of the model, inference time, or energy consumption are not discussed in this paper. Recently, Venkatachalam et al.[36] proposed a system for real-time person identification that also employs inertial (accelerometer and gyroscope) information. They proposed a 2D CNN that processes FFT features extracted from temporal sensor signals. The deployment is performed on a mobile phone. Experiments were very limited, employing only one person, and no optimizations were applied. Tiñini et al.[37] proposed the deployment of a vision-based gait recognition system using an OAK-D camera and a Jetson Nano. They deploy a MobileNetV2 model to detect people, a U-Net model to segment them, and construct a modified Gait Energy Image that is classified using LDA. They compared three U-Net models that varied in size, but there was no optimization of any of the models to adapt them to the edge device. Later, in[38] the LDA classifier is replaced by a custom CNN to improve accuracy, but no further optimizations are performed. Finally, in[2] an extensive study of visual gait recognition models is performed. They apply both quantization using FP16 and INT8 data representations and structural pruning to reduce computational requirements, while batching is applied to increase the performance of deployed models. However, they use only a single objective, that is, EDP (energy-delay product) minimization, for the model selection process. In addition, they did not consider the impact of inference concurrency on performance.

In contrast to previous works that focused on limited implementation scenarios or lacked comprehensive optimization strategies, our approach presents a fully optimized system. We performed a multiobjective evaluation to balance the energy consumption, latency, and accuracy through an extensive analysis of pruning and quantization techniques. Moreover, unlike prior studies, our work introduces a novel model selection framework based on Pareto optimality and investigates the impact of batching and concurrency, which have been largely overlooked in earlier research.

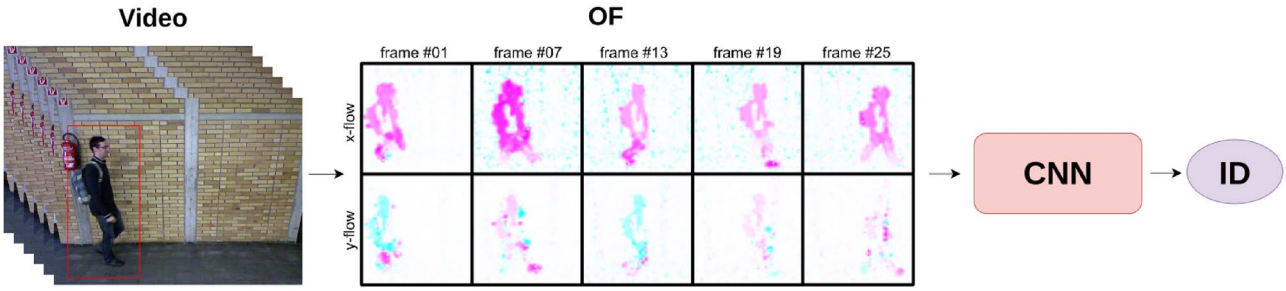


Fig. 2. Input data. The input data is a set of 25 consecutive optical flow maps extracted from a video. The region of the subject is input to the models. Finally, we identify every person. Data belonging to the publicly available TUM dataset[40].

Model	N. layers	N. parameters	N. activations	GFLOP
2D-CNN	11	6.83M	0.68M	2.03
3D-CNN	8	9.90M	1.86M	2.69

Table 1. Models comparison. The number of layers, parameters, activations, and floating-point operations (in GFLOP) is reported in each column.

Framework description

In this section, we first present the two models selected for gait recognition. We then introduce the input data used in our experiments, and we describe a framework to select the best parameters for both models. An overview of the gait recognition pipeline is shown in Fig. 2.

Gait recognition models

In this work, we selected two state-of-the-art gait recognition approaches for the given environment based on two key design elements: 2D and 3D convolutions. This way, the first model (2D-CNN) focuses on the spatial information using 2D convolutions whose execution is extremely optimized, and the second model (3D-CNN) focuses on the spatio-temporal information using 3D convolutions that build more robust descriptors but are less optimized and have more parameters. Note that, although more recent approaches exist, most of them are significantly more complex and are designed for multiview gait recognition scenarios, where viewpoint variation is a major challenge. In contrast, in our scenario, the gait is captured from a lateral view, and complex models are not required. Therefore, we selected models that provide top-tier performance under these specific conditions.

2D-CNN

Model based on 2D convolutions, which was introduced in[39]. It uses a linear pipeline of convolutional and pooling layers, which is the most traditional architecture for this type of problem. The model comprises four Conv2D blocks, each containing a Conv2D layer and a Max Pooling layer. As the layer depth increases, the number of convolutional filters also increases, meaning deeper layers have more filters. In addition, the model includes three fully connected layers. Further details about the architecture can be found in Table 1. This model has 11 layers and a considerably lower number of parameters than the other model. Moreover, it has a limited number of activations (0.68M), which is the smallest along with the total number of floating-point operations of the model (2.03 GFLOP). Therefore, this model does not require substantial computational resources.

3D-CNN

Model described in[39] that uses 3D convolutions to capture temporal information from videos. It comprises seven Conv3D layers, and the number of filters in each layer increases as we move deeper into the architecture. The model ends with a fully connected layer. This model helps evaluate the performance of 3D convolutions, which are less optimized than 2D convolutions across all frameworks and hardware devices. As indicated in Table 1, it contains a large number of parameters, which reduces the number of activations. However, the number of GFLOP is relatively low compared to the number of parameters, likely due to the cuDNN implementation of 3D convolutions.

Input data

In both models, optical flow (OF)[41] is used as input. OF is a term used to describe the motion pattern in a scene that is produced by the relative motion between an observer and the scene, observed at two different instants in time. OF is a representation that focuses on describing a subject through a set of local and subtly varying motions rather than appearance. Therefore, OF has shown impressive results in the characterization of gait[39]. It is divided into two channels, one for the y-axis and the other for the x-axis, where most of the gait motion flow is concentrated. The input to both models is a stack of 25 consecutive optical flow maps containing a single subject

to be identified. Thus, the region of the subject is extracted and resized to 60×60 and input into our models. Note that, if multiple people are present in the scene, we can extract one region per person to identify all of them.

Model selection

In this work, the selection of the best parameters for a gait inference model is proposed as a multi-objective optimization problem:

$$\min_{x \in X}(f_a(x), f_l(x), f_e(x)) \tag{1}$$

where the objectives, f_a, f_l, and f_e correspond to the accuracy, latency, and energy consumption of an inference using the model, respectively, and X is the feasible set of decision vectors. In particular, we consider only these three objectives; however, other objectives, such as inference throughput and model size, could also be considered. If some objective function is to be maximized, for example accuracy, we minimize its negative value because it is equivalent to maximizing it.

The feasible set of decision vectors in Equation 1 is related to the decision space and includes parameters that can be varied to obtain a particular model. In "Related work" we presented some techniques that can be used to form this decision space. Although it is not explicitly stated, this vector is subject to constraints that define its limits. Specifically, we use the pruning factor, GPU frequency, and data precision to form the decision vector.

In multi-objective optimization problems, all objectives are optimized simultaneously, and there is usually not a single viable optimal solution for all optimization objectives; thus, a set of suboptimal solutions is sought to select one of them. This set is typically formed with Pareto optimal solutions, i.e., solutions that cannot be improved for any objective without degrading at least one of the other objectives. This set is called the Pareto front and can be easily computed for decision spaces with a finite set of alternatives, as occurs in this work. In a production environment, there are generally some requirements that must be met (such as a maximum response time), so any Pareto front solution that meets these constraints can be selected for deployment.

Experimental results

All experiments in this work were conducted using two NVIDIA Jetson platforms with very different computational capabilities, namely Jetson Orin Nano and Jetson AGX Orin, which we will refer to as Nano and Orin, respectively. The main specifications of both devices are listed in Table 2. We also used NVIDIA TensorRT[42], a library designed to optimize deep learning models for NVIDIA devices. It features a deep learning inference optimizer and runtime that facilitate the creation of inference models with low latency and high throughput. This is achieved using quantization, layer fusion, and memory bandwidth optimization techniques. The optimized models can be deployed in hyperscale data centers, embedded devices, and automotive platforms. In the rest of this section, we first discuss the optimization space for each model. Next, we obtain the Pareto front on each device and explore different scenarios for concurrent deployment.

Training details

The training of both 2D-CNN and 3D-CNN architectures was performed using Stochastic Gradient Descent (SGD) with a momentum factor of 0.9. A weight decay of 5×10^{-6} was applied, and the initial learning rate was set to 10^{-3}. When the validation error got stuck, the learning rate was divided by 0.2. Each training epoch processed mini-batches of 150 samples, and training was continued for 100 epochs using the cross-entropy loss function. Implementations were done using TensorFlow 2.12, TensorRT 10.3 and cuDNN 9.3.

The optimization space for each model

The parameters that form the decision vector of the proposed optimization space are the data precision, pruning factor, and GPU frequency. In this subsection, we compute the feasible range of these parameters. We use the TUM dataset[40] to compute the accuracy of each version of both models.

Precision is related to the data type used to represent and process the weights of the model. In Jetson devices, it is possible to use FP32, FP16, or INT8 data types. TensorRT supports mixed precision inference in four different modes: in FP32, all layers are executed using FP32; in FP16, a layer can either run in FP32 or FP16 based on the fastest execution time; in INT8, it can equivalently use FP32 or INT8, and in BEST mode, layers can be executed using all supported precisions. We used the full precision model to obtain FP32 and FP16 TensorRT versions and a post-training quantized version of the model to obtain the INT8 and BEST versions.

Our filter-wise pruning uses L_pnorm, a common metric for determining which filters can be removed[18] in order to eliminate filters that less affect the accuracy of the classification. For us, L_pnorm is computed as $L_pnorm = (\sum_{i=1}^{n} |x_i|^p)^{1/p}$, with $p = 1$, where x is a vector, n is the number of filter elements, and p is a real number. In this way, we calculate the L_1norm for each filter of each convolutional layer and then, remove filters

Board name	CPU	GPU	AI performance	Power
Jetson Orin Nano	6x NVIDIA Arm Cortex A78AE v8.2@1.5GHz	1024-core Nvidia Ampere with 32 Tensor Cores@625MHz	67 TOPS	7–25 W
Jetson AGX Orin	12x NVIDIA Arm Cortex A78AE v8.2@2.2GHz	2048-core Nvidia Ampere with 64 Tensor Cores@1.3GHz	275 TOPS	15–60 W

Table 2. Main characteristics of two modern embedded computing boards with heterogeneous architecture. AI performance is measured in trillions of operations per second.

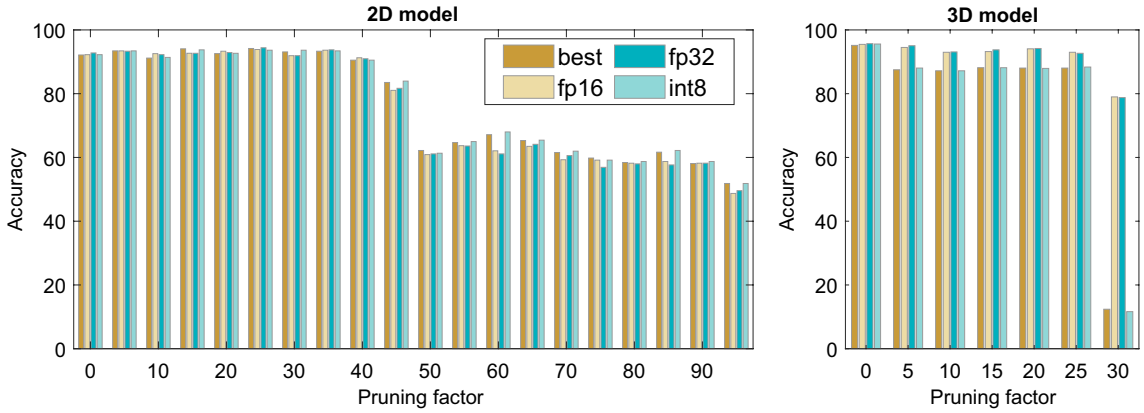


Fig. 3. Accuracy values for different pruning factors. The left side shows accuracy results for the 2D model, while the right side depicts results of accuracy for 3D model. Each color represents a different data precision.

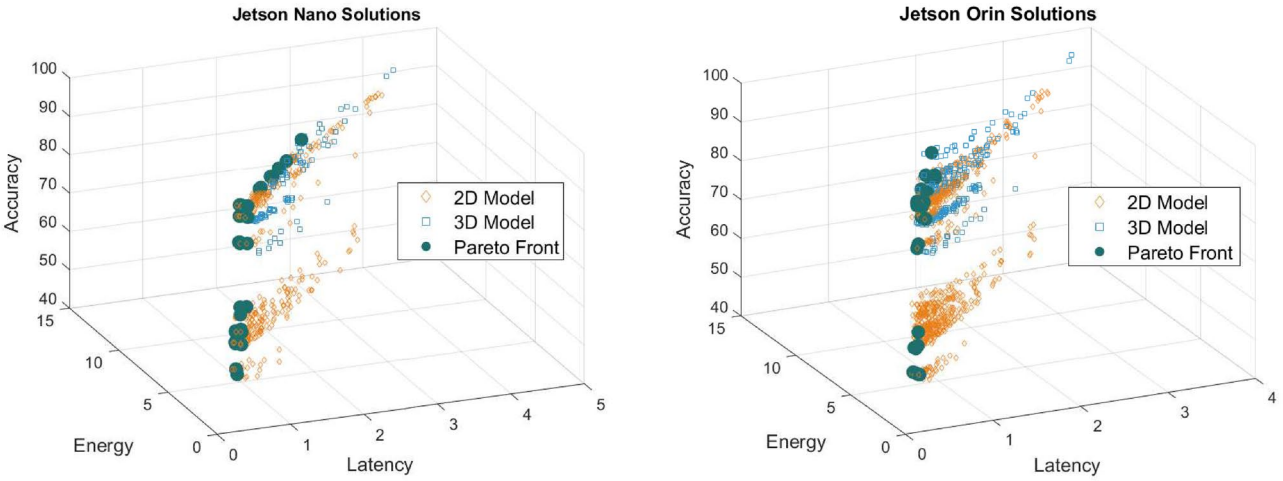


Fig. 4. Optimization space based on accuracy, latency and energy consumption. The graph on the left corresponds to a Nano device, while the graph on the right corresponds to an Orin device. Each rhombus represents a 2D model based on its accuracy, latency, and energy consumption, whereas squares represent 3D models. Finally, the solid dots form the Pareto front and correspond to the non-dominated solutions of the objective function.

whose L_1 value is lower than a predefined percentile threshold. With the aim of not removing entire layers and preserving the architecture of the original models, we ensure that each layer has at least one filter. Finally, once the less important filters have been removed, we perform a fine-tuning step to stabilize the model and improve its behavior.

We applied pruning with percentile thresholds in increments of 5% as long as the training converged. We denote these percentile thresholds as pruning factors, *PF*, which range from 0 to 95 for the 2D model and from 0 to 30 for the 3D model. The accuracy of the 2D model drops below 60% with factors greater than 50, whereas the 3D model supports lower pruning factors (up to 30). Nevertheless, the accuracy was similar for all valid pruning factors when using either floating point or integer precision. See Figure 3 for further details.

Finally, in Jetson devices, it is possible to vary the working frequencies of the CPU, GPU, and memory. The frequency selection directly affects the latency and energy consumption of the inferences. In this work, we evaluated the impact of changing the GPU frequency. In Orin devices, the frequency can be selected from a predefined set, with 11 different values ranging between approximately 306MHz and 1300MHz, while in Nano devices there are 5 values between approximately 306MHz and 624MHz.

Thus, the optimization space consists of $4 \cdot 20 \cdot 11 = 880$ 2D models and $4 \cdot 7 \cdot 11 = 308$ 3D models in the Orin, and $4 \cdot 20 \cdot 5 = 400$ 2D models and $4 \cdot 7 \cdot 5 = 140$ 3D models in the Nano. Figure 4 plots the latency, energy, and accuracy of a subset of these models. As expected, the Orin device is more efficient than the Nano device. It can run at nearly twice the frequency to obtain an average latency speed-up of over 1.65, while the energy consumption remains almost equal. Accuracy values range between 48.71% and 95.68%, latency varies between 0.4 ms and 3.7 ms, and energy ranges between 1.15 and 14.15 mJ. There is no model with the best

overall accuracy, latency, and energy; thus, Pareto optimal solutions should be obtained to choose one of them for deployment based on some constraints.

Pareto front selection

The non-dominated solutions of the objective function form the Pareto front. A solution is non-dominated if there is no other solution that is better in all objectives, that is, there is no feasible solution that is better in one objective and at least as good in all other objectives. Since the solution space is small (around 1000 solutions), it is possible to search for non-dominated solutions by comparing each solution with the rest of the space. For more complex cases, algorithms can be implemented to obtain the Pareto front in $O(n \log n)$ time, where n is the number of solutions in the space[43]. In the Nano device we can identify 33 non-dominated solutions in our optimization space that formed the Pareto front, while in the Orin device, there were 32 solutions. These solutions are represented by solid green dots in Fig. 4.

Figure 5 plots the latency, accuracy, and energy consumption of each solution in the Pareto front. Most solutions correspond to the 2D model, which is shown in light brown in the figure, whereas the solutions corresponding to the 3D model are shown in teal. They are ordered from lowest to highest latency so that, for example, in the left plot, bar number 2 corresponds to the latency, accuracy, and energy of one of the 2D models, while bar number 31 shows the values of one of the 3D models.

Other works, like [2], use the EDP (Energy Delay Product) to select the best model among all models with an accuracy greater than 90%. Solutions with this minimum EDP value for both the 2D and 3D models and the Nano and Orin devices correspond to one of the Pareto front solutions. Thus, the proposed framework is capable of obtaining the best solutions, including those with the minimum EDP, without imposing any restriction on accuracy, energy, or latency.

Scenarios for concurrent inferences

In our motivating example (see Fig. 1), several video surveillance cameras monitor the flow of people through the entrance hall of a building. If several subjects are present in the scene, several inferences (i.e. one per subject) must be satisfied simultaneously. Current embedded platforms such as Nano and Orin, which offer important hardware resources in the form of RAM and computational capabilities, can take advantage of simultaneous inferences to increase throughput. In this Section, we explore two mechanisms to efficiently perform inferences using sample batching and streaming execution.

The use of a batch of samples involves performing inference on multiple samples simultaneously. Thus, batch processing exposes more parallelism to the underlying hardware, thereby enabling better resource utilization. In addition, by employing streaming execution, different samples can be inferred concurrently. Thus, latency caused by host memory transfers to/from device memory or synchronization primitives during one inference can be hidden by concurrent execution of another. In addition, concurrency can simultaneously execute kernels from different inferences when the use of computing resources is low. For this purpose, we employed Nvidia's profiling tool to explore the occupancy achieved by the kernels of each model to determine the possibility of launching several inferences concurrently. In the Nano device, 6 of the 8 kernels of the 2D model achieved an occupancy below 17% and only one kernel achieved an occupancy of 75%. In the 3D model 6 of the 10 kernels have a low occupancy, below 15%, and only 2 kernels reach 90% occupancy. The average occupancy of the 2D and 3D model is, respectively, 22% and 34%. These figures for the Orin device are lower, with average values of 16% and 30%, respectively. In all cases the reason for the low occupancy was the small size of the launch grid. Only a few blocks are required to compute most of the network layers of each model; thus, it is possible to launch

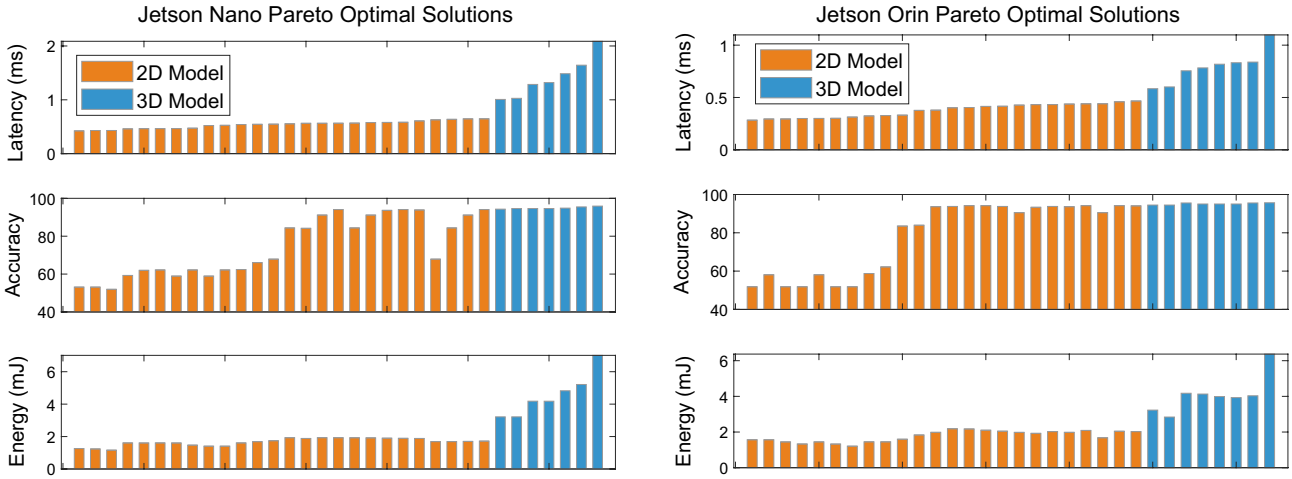


Fig. 5. Pareto optimal solutions. The graphs on the left correspond to a Nano device, while the graphs on the right correspond to an Orin device. From top to bottom, the Y-axis shows the latency, accuracy, and energy consumption of each solution.

additional kernels belonging to other inferences. Note that both alternatives to speed up inferences, that is batch processing and concurrent execution, are complementary and can be applied simultaneously.

In Fig. 6, we show the implementation details of our solutions for multiple people detection in the scene (multidetection relies on a people detector and tracker, with implementation details to be explored in future work rather than in this paper). As explained in "Input data", each inference model requires an input of 25 consecutive optical flow maps of the same person (sample size). In inference time, we build this input by pushing new optical flow maps into FIFO queues with a storage capacity of 25 optical flow maps. Thus, each detection generates a tuple $ROI_t^l = (ROI, t, l)$, where *ROI* is the optical flow of the region where the person has been localized, *t* is the detection timestamp, and *l* is the label that identifies the same detected person during tracking. This information is pushed into the FIFO queue containing consecutive *ROIs* belonging to the same subject. The time interval between consecutive *ROIs* is dependent on the frame rate of the camera. On the left side of Fig. 6 (CPU box), we can see a scenario in which *M* different people were detected. Thus, *M* FIFO queues holding consecutive *ROIs* of *M* people are used. As previously indicated, each *FIFO* queue holds 25 *ROIs*. On the right side of the figure (GPU box), the content of several *FIFO* queues is grouped into batches. Then, each batch is launched using different threads and CUDA streams such that concurrent inferences on the same model are executed on different instances of the same model. The batch size and concurrency degree can vary according to the throughput and energy consumption requirements.

Experiments with concurrent inferences

To evaluate how concurrency and batch processing affect gait recognition models, we tested two scenarios:

- Maximum precision: prioritizing accuracy over speed, selecting the unpruned 3D model (`3D0best`) with optimal quantization and maximum clock frequency for both Nano and Orin devices.
- High throughput: targeting >90% accuracy while maximizing inferences per second, choosing, in the Orin device, a 2D model (`2D30best`) with pruning 30, best quantization and max frequency and, in the Nano device, a 2D model (`2D40int`), with pruning 40, INT8 quantization and max frequency.

Models were deployed on an NVIDIA Triton server[44] across both devices, with a custom inference client co-located on each platform to minimize latency. While the measured throughput and latency include client-server communication, the values are kept low due to the on-device deployment.

The models were evaluated by calculating the achieved throughput (measured in inferences per second) for various values of batch size and concurrency. In addition, we measured the energy consumption in millijoules per inference for each configuration. This was achieved by integrating the product of the instantaneous electrical current (in milliamperes) and the voltage. The resulting value was divided by the number of inferences made.

In Fig. 7, the throughput (red) and energy consumption (green) of the 3D0best model on the Nano device are analyzed for batch sizes (b) ranging from 1 to 32 and concurrency levels (c) from 1 to 16, both in increments of 4. The results show that some concurrency is essential to achieve high throughput. For instance, at c = 1, throughput peaks at approximately 350 inferences per second, while increasing concurrency with b = 1 raises throughput to 822 inferences per second. Resource saturation occurs in the reddest area of Fig. 7a, where further parallelism no longer improves throughput. Regarding energy consumption, Fig. 7b reveals that batching reduces energy per inference for b values between 1 and 12, while energy remains stable across c values from 4 to 16 for a given batch size. We aim to evaluate the proposed model's performance in a multi-person gait identification scenario, focusing on maximizing throughput while minimizing energy consumption per inference. Based on Fig. 7, the optimal configuration is c = 8 and b = 16, which achieves a throughput of 1063 inferences per second.

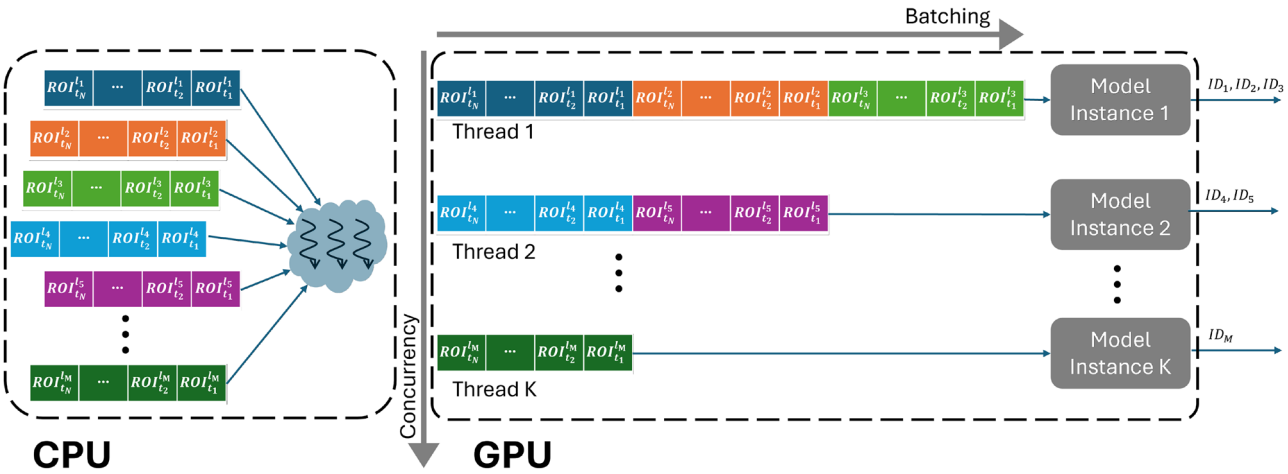


Fig. 6. Multi-tennant server. The CPU threads in the edge server handle the *ROIs* of *M* multiple subjects and push them into *FIFO* queues. These queues are grouped into *K* dynamic batches, and concurrent inferences using CUDA streams are executed by launching *K* model instances in the GPU.

	b=1	b=4	b=8	b=12	b=16	b=20	b=24	b=28	b=32
c=1	184.8	255.3	284.8	286.6	289.7	291.5	319.9	311.1	325.3
c=4	690.7	981.5	868.2	946.4	1035.2	1040.9	1051.7	1069.9	1027.3
c=8	743.6	995.7	1015.7	1040.4	**1063.7**	1053.1	1067.7	1079.3	1086.0
c=12	811.3	995.7	1046.7	1038.4	1061.0	1057.5	1066.4	1083.9	1089.6
c=16	822.6	987.2	1059.2	1042.4	1055.7	1064.2	1070.4	1085.5	1078.8

(a) Throughput in inferences per second

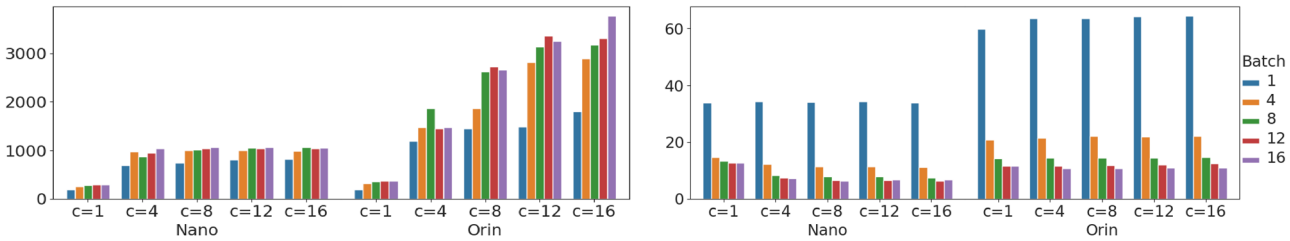
	b=1	b=4	b=8	b=12	b=16	b=20	b=24	b=28	b=32
c=1	33.87	14.71	13.27	12.82	12.70	12.44	12.74	12.76	12.12
c=4	34.19	12.18	8.38	7.50	7.24	7.46	7.50	7.56	7.46
c=8	33.99	11.34	7.88	6.51	**6.31**	6.91	6.97	7.04	7.28
c=12	34.19	11.42	7.94	6.49	6.67	7.14	7.16	7.58	6.99
c=16	33.79	11.10	7.54	6.25	6.67	7.14	6.93	7.12	7.16

(b) Energy in mJ per inference

Fig. 7. Throughput and energy in nano device: (**a**) throughput (in red), expressed in inferences per second, and (**b**) energy, indicated in mJ per inference (in green), achieved by the Nano in the `3D0best` model for different combinations of batching and concurrency. The values highlighted in bold indicate a suitable selection of batch size and concurrency (b = 16 and c = 8) that achieve high throughput and low energy consumption.

Device	Model	b	c	Through. inf./s	Energy mJ/inf.	Power W	Num. people	Acc.
Nano	`3D0best`	16	8	1063	6.31	6.7	42	95.9
	`2D40int`	28	8	2450	3.86	9.4	98	91.1
Orin	`3D0best`	20	16	4260	9.71	41.3	170	95.9
	`2D30best`	12	28	4710	6.77	31.9	188	93.2

Table 3. A selection of batch sizes (b) and concurrency levels (c) for the different devices and models that can identify several simultaneous subjects, expending low energy consumption.



(a) Inferences per second. **(b)** Energy per inference (mJ).

Fig. 8. 3D0best model in Nano and Orin. We compare the throughput and energy consumption of the 3D0best model for both devices.

Considering the camera's recording rate of 25 frames per second per subject, this setup enables the Nano device to process up to $\lfloor 1063/25 \rfloor$ frames, equivalent to 42 subjects simultaneously using the `3D0best` model.

Table 3 summarizes the evaluation of 2D and 3D models on the Nano and Orin devices, highlighting configurations that maximize throughput while minimizing energy consumption. It also includes mean power consumption, maximum simultaneous people identified, and precision on the TUM-GAID dataset. 2D models outperform 3D models in throughput, with the `2D40int` model recognizing nearly 98 people on the Nano and the `2D30best` model recognizing about 190 on the Orin. Energy-wise, 2D models consume less due to lower computational complexity. For example, the `3D0best` model uses 6.31 mJ on Nano versus 9.71 mJ on Orin, a 35% savings. Despite Orin's throughput being 4.43 times higher, its average power consumption is 6.16 times greater than Nano's. Deploying five Nano devices could surpass Orin's throughput while consuming less energy, making Nano preferable when energy efficiency is critical.

In Fig. 8, we compare the performance of 3D0best across two devices, focusing on batching and concurrency. The Orin device consistently achieves higher throughput, with speedup increasing as concurrency rises, while the Nano's concurrency benefits stabilize at c = 4. Regarding energy consumption per inference, batch sizes above 1 significantly reduce energy usage for both devices. Energy-per-inference remains stable across concurrency levels but decreases with larger batch sizes. Overall, increasing batch size reduces energy consumption, and higher concurrency boosts throughput. The Nano demonstrates lower energy consumption than the Orin under similar configurations, often achieving values around 6 mJ compared to Orin's 10 mJ.

Conclusion

The main objective of this work was to study the deployment of two state-of-the-art gait recognition models on two low-power embedded platforms from NVIDIA. We selected two models (one based on 2D convolutions

and another one based on 3D convolutions) that provide top results for lateral viewpoint gait recognition. While more recent and complex models have been proposed in the literature, they are mainly designed for multiview scenarios and tend to perform worse in single-view (lateral) contexts due to their added complexity. In our experiments, more than one thousand model versions were generated by applying different quantization representations, pruning factors, and working frequencies. Then, a multiobjective optimization technique is proposed to find optimal model versions belonging to Pareto's front of the configuration space. The optimization searches for the most efficient configurations in terms of accuracy, latency, and energy consumption. Finally, the selected model versions are evaluated using techniques such as batching and concurrent execution, which are necessary to develop multi-subject real-time gait recognition solutions. Different values for batch size and concurrency are explored, and tradeoffs between throughput and energy consumption are discussed. Finally, we have demonstrated that, depending on the platform and model, between 42 and 188 people can be managed simultaneously via batching and concurrency techniques. Our solution can scale, if required, by adding more embedded platforms. Regarding energy consumption, we have shown that 2D models consume less energy per inference than the 3D models version in the same device and that the Nano device typically consumes less energy per inference than the Orin device. In contrast, accuracy achieved by the 3D models is higher than that obtained by the 2D models. This is an important aspect to consider if security is an issue. In future work, we plan to deploy multiview models able to cope with different subject views. Also, preprocessing steps, i.e., optical flow computation and tracking will be implemented to build a complete recognition system. In addition, we intend to use model simplification employing knowledge distillation techniques to reduce the computational requirements of the model and increase the throughput even more.

Data availability

Gait models and source code are available at https://github.com/PaulaRuizB/rt_gait_edge/, and from the corres ponding author on reasonable request.

Received: 24 December 2024; Accepted: 13 May 2025
Published online: 02 June 2025

References

1. Yedilkhan, D., Kyzyrkanov, A. E., Kutpanova, Z. A., Aljawarneh, S. & Atanov, S. K. Intelligent obstacle avoidance algorithm for safe urban monitoring with autonomous mobile drones. *J. Electron. Sci. Technol.* **22**, 100277. https://doi.org/10.1016/j.jnlest.2024.100277 (2024).
2. Ruiz-Barroso, P., Castro, F. M., Delgado-Escaño, R., Ramos-Cózar, J. & Guil, N. High performance inference of gait recognition models on embedded systems. *Sustain. Comput. Inform. Syst.* **36**, 100814 (2022).
3. Zhou, Z. et al. Edge intelligence: Paving the last mile of artificial intelligence with edge computing. *Proc. IEEE* **107**, 1738–1762. https://doi.org/10.1109/JPROC.2019.2918951 (2019).
4. Jan, F., Min-Allah, N., Agha, S., Usman, I. & Khan, I. A robust iris localization scheme for the iris recognition. *Multimed. Tools Appl.* **80**, 4579–4605. https://doi.org/10.1007/s11042-020-09814-5 (2021).
5. West, B. J. & Scafetta, N. Nonlinear dynamical model of human gait. *Phys. Rev. E* **67**, 051917 (2003).
6. Scafetta, N., Marchi, D. & West, B. J. Understanding the complexity of human gait dynamics. *Chaos Interdiscip. J. Nonlinear Sci.* **19**, 026108 (2009).
7. Gupta, G., Pequito, S. & Bogdan, P. Re-thinking EEG-based non-invasive brain interfaces: Modeling and analysis. In *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. 275–286 (IEEE, 2018).
8. Xue, Y., Rodriguez, S. & Bogdan, P. A spatio-temporal fractal model for a cps approach to brain-machine-body interfaces. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 642–647 (IEEE, 2016).
9. Di Stasi, S. L., Logerstedt, D., Gardinier, E. S. & Snyder-Mackler, L. Gait patterns differ between ACL-reconstructed athletes who pass return-to-sport criteria and those who fail. *Am. J. Sports Med.* **41**, 1310–1318 (2013).
10. Sepas-Moghaddam, A. & Etemad, A. Deep gait recognition: A survey. *IEEE Trans. Pattern Anal. Mach. Intell.* (2022).
11. Chao, H., He, Y., Zhang, J. & Feng, J. Gaitset: Regarding gait as a set for cross-view gait recognition. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2019).
12. Shiraga, K., Makihara, Y., Muramatsu, D., Echigo, T. & Yagi, Y. Geinet: View-invariant gait recognition using a convolutional neural network. In *International Conference on Biometrics (ICB)*. 1–8 (2016).
13. Castro, F. M., Marín-Jiménez, M. J., Guil, N. & de la Blanca, N. P. Automatic learning of gait signatures for people identification. *IWANN* **10306**, 257–270 (2017).
14. Delgado-Escaño, R., Castro, F. M., Cózar, J. R., Marín-Jiménez, M. J. & Guil, N. An end-to-end multi-task and fusion CNN for inertial-based gait recognition. *IEEE Access* **7**, 1897–1908 (2018).
15. Shen, C., Yu, S., Wang, J., Huang, G. Q. & Wang, L. A comprehensive survey on deep gait recognition: Algorithms, datasets, and challenges. *IEEE Trans. Biometrics Behav. Identity Sci.* 1–1. https://doi.org/10.1109/TBIOM.2024.3486345 (2024).
16. Shi, W. & Dustdar, S. The promise of edge computing. *Computer* **49**, 78–81. https://doi.org/10.1109/MC.2016.145 (2016).
17. Mathew, M., Desappan, K., Kumar Swami, P. & Nagori, S. Sparse, quantized, full frame cnn for low power embedded devices. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops* (2017).
18. Liang, T., Glossner, J., Wang, L., Shi, S. & Zhang, X. Pruning and quantization for deep neural network acceleration: A survey. *Neurocomputing* **461**, 370–403. https://doi.org/10.1016/j.neucom.2021.07.045 (2021).
19. Jaiswal, B. & Gajjar, N. Deep neural network compression via knowledge distillation for embedded applications. In *NUiCONE*. Vol. 1–4. https://doi.org/10.1109/NUICONE.2017.8325620 (2017).
20. Kochura, Y. et al. Batch size influence on performance of graphic and tensor processing units during training and inference phases. arXiv arXiv:1812.11731 (2019).
21. Kim, M. Guaranteeing that multi-level prioritized DNN models on an embedded GPU have inference performance proportional to respective priorities. *IEEE Embedded Syst. Lett.* 1–1 . https://doi.org/10.1109/LES.2021.3129769 (2021).
22. Minhas, U. I. et al. Increased leverage of transprecision computing for machine vision applications at the edge. *J. Signal Process. Syst.* **94**, 1101–1118. https://doi.org/10.1007/s11265-022-01784-1 (2022).
23. Scheidegger, F., Benini, L., Bekas, C. & Malossi, A. C. I. Constrained deep neural network architecture search for Iot devices accounting for hardware calibration. *Adv. Neural Inf. Process. Syst.* **32** (2019).
24. Mazzia, V., Khaliq, A., Salvetti, F. & Chiaberge, M. Real-time apple detection system using embedded systems with hardware accelerators: An edge AI application. *IEEE Access* **8**, 9102–9114 (2020).

25. Jeong, E. J., Kim, J., Tan, S., Lee, J. & Ha, S. Deep learning inference parallelization on heterogeneous processors with tensorrt. *IEEE Embedded Syst. Lett.* (2021).
26. Nguyen, H.-H., Tran, D. N.-N. & Jeon, J. W. Towards real-time vehicle detection on edge devices with NVIDIA JETSON tx2. In *2020 IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia)*. 1–4 (IEEE, 2020).
27. Wang, Z., Li, C. & Wang, X. Convolutional neural network pruning with structural redundancy reduction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14913–14922 (2021).
28. Luo, J.-H. & Wu, J. Autopruner: An end-to-end trainable filter pruning method for efficient deep model inference. *Pattern Recognit.* **107**, 107461 (2020).
29. Liu, Z. et al. Post-training quantization for vision transformer. *NeurIPS* **34**, 28092–28103 (2021).
30. Park, E., Yoo, S. & Vajda, P. Value-aware quantization for training and inference of neural networks. In *ECCV*. 580–595 (2018).
31. Zhao, K. et al. Distribution adaptive int8 quantization for training CNNs. *Proc. AAAI Conf. Artif. Intell.* **35**, 3483–3491 (2021).
32. Goncharenko, A., Alyamkin, S., Denisov, A. & Terentev, E. Winning solution on LPIRC-ll competition. In *CVPR Workshops*. 10–16 (2019).
33. Zhang, D. Y., Vance, N., Zhang, Y., Rashid, M. T. & Wang, D. Edgebatch: Towards AI-empowered optimal task batching in intelligent edge systems. In *Proceedings - Real-Time Systems Symposium*. 366 – 379. https://doi.org/10.1109/RTSS46320.2019.00040 (2019).
34. Subedi, P., Hao, J., Kim, I. K. & Ramaswamy, L. Ai multi-tenancy on edge: Concurrent deep learning model executions and dynamic model placements on edge devices. In *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. 31–42. https://doi.org/10.1109/CLOUD53861.2021.00016 (2021).
35. Zeng, X., Zhang, X., Yang, S., Shi, Z. & Chi, C. Gait-based implicit authentication using edge computing and deep learning for mobile devices. *Sensors* **21**. https://doi.org/10.3390/s21134592 (2021).
36. Venkatachalam, S. et al. Realtime person identification via gait analysis. arXiv preprint arXiv:2404.15312 (2024).
37. Tiñini Alvarez, I. R., Sahonero–Alvarez, G., Menacho, C. & Suarez, J. Exploring edge computing for gait recognition. In *2021 4th International Conference on Bio-Engineering for Smart Technologies (BioSMART)*. 01–04. https://doi.org/10.1109/BioSMART54244.2021.9677840 (2021).
38. Conchari, C., Sahonero-Alvarez, G., Mollocuaquira, R. & Salazar, E. Distributed edge computing for appearance-based gait recognition. In *2024 IEEE Andescon*. 1–6 (2024).
39. Castro, F. M., Marín-Jiménez, M. J., Guil, N. & de la Blanca, N. P. Multimodal feature fusion for CNN-based gait recognition: an empirical comparison. *Neural Comput. Appl.* 1–21 (2020).
40. Hofmann, M., Geiger, J., Bachmann, S., Schuller, B. & Rigoll, G. The tum gait from audio, image and depth (GAID) database: Multimodal recognition of subjects and traits. *J. Vis. Commun. Image Represent.* **25**, 195–206 (2014).
41. Horn, B. K. & Schunck, B. G. Determining optical flow. *Artif. Intell.* **17**, 185–203 (1981).
42. TensorRT. https://developer.nvidia.com/tensorrt (2024).
43. Kung, H. T., Luccio, F. & Preparata, F. P. On finding the maxima of a set of vectors. *J. ACM* **22**, 469–476. https://doi.org/10.1145/321906.321910 (1975).
44. Nvidia Triton Inference Server (2024).

Acknowledgements

This work has been supported by European Union Next Generation (TSI-069100-2023-0013), the Junta de Andalucía of Spain (UMA20-FEDERJA-059), the Ministry of Education of Spain (PID2022-136575OB-I00), and the University of Málaga (B1-2022_04).

Author contributions

All authors contributed equally to this work. PRB & FC built the gait recognition models and their pruned versions, JGL migrated them to the Jetson devices and implemented the multi-objective optimization framework, and NG designed and conducted the experiments with the Triton server on the Jetson devices. All authors discussed the results and implications and commented on the manuscript at all stages.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to J.M.G.-L.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025